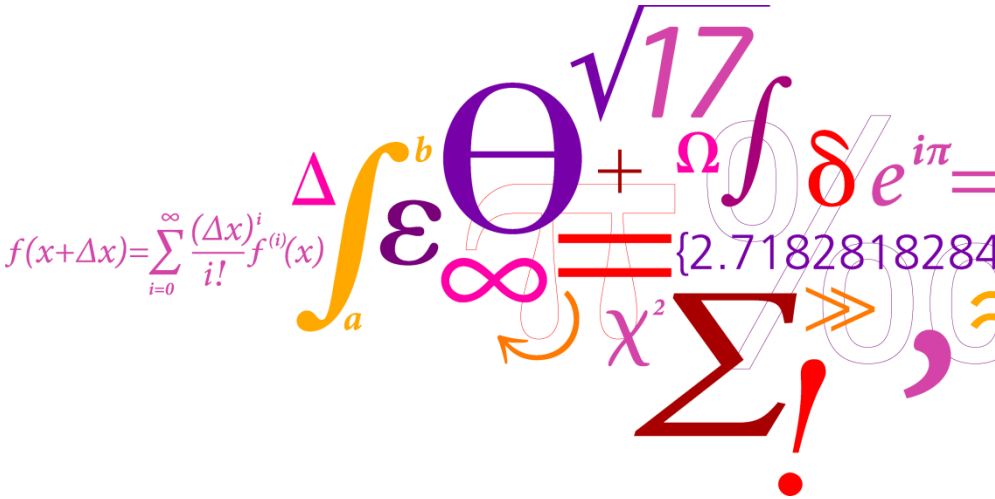
Building Dependable Systems

Test and Debugging



DTU Electrical Engineering
Department of Electrical Engineering

Introduction

Test

Debugging

Testing

Tests are performed to show that system works as intended.

Tests are good at detecting errors but cannot guarantee absence of errors

Tests normally consists of a number of test cases with specified

- Test inputs
- Test conditions
- Expected system outputs

Exhaustive Testing

An exhaustive test will test for

All possible inputs

All possible conditions

All outputs

A successful exhaustive test will guarantee the system to be error free

Exhaustive testing is not possible for most real systems.

Exhaustive Testing

Given a system with 6 binary inputs and one binary output

This gives 2^6 =64 different input combinations

In this case exhaustive testing is possible

Give a system with two 16 bit integer inputs and one 32 bit integer output

e. g. multiplication:

This gives $2^{16} * 2^{16}$ =4.3 10^9 different input combinations

Exhaustive testing is nearly impossible

A double has 2^64 =1.84 10^{19} different values

i.e. exhaustive tests of even simple math functions are excluded

Black Box Testing

Black box testing is test without knowing the internal structure of the tested system

If exhaustive test is possible the method works well

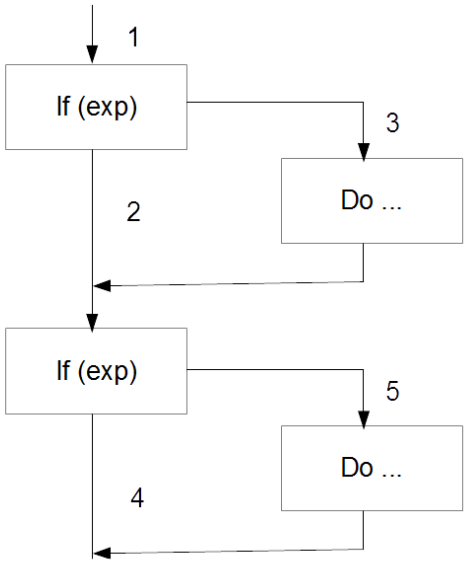
For most systems it is inefficient as a lot of the tests probably will test the same part of the system

White Box Testing

White box testing assumes knowledge about the inner structure of the system

This makes it possible to design of efficient test cases and test data that will test most parts of the system

White Box Test Methods



- Statement testing
- Branch testing
- Path testing

Testing a Quadratic Equation Solver

Given

$$A\,x^2 + B\,x + C = 0$$

If you only can use four tests which will you choose?

Testing a Quadratic Equation Solver

Given

$$A\,x^2 + B\,x + C = 0$$

Solution:

$$D = B^2 - 4\,AC$$

$$x = \frac{-B+\sqrt{D}}{2A} \quad \text{or} \quad x = \frac{-B-\sqrt{D}}{2A}$$

- D > 0
- D = 0
- D < 0
- A = 0

Testing a Coordinate Transformation

The transformation between a task space coordinate system and the robot base coordinate system is found by:

- Move the robot endeffector to origo P0 of the task system and note the robot base coordinates
- Move the robot endeffector to a point P1 on the x-axis of the task system and note the robot base coordinates
- Move the robot endeffector to a point P2 on the y-axis of the task system and note the robot base coordinates
- Calculate the transformation between the task system defined by the vectors P0P1 and P0P2
- Test the transformation with the taskpoint (0.1, 0.1,0)

Testing a System

A system normally consists of a number of sub systems or components.

The system is tested by testing subsystems individually first and then together

Bottom up testing:

Start with the lowest level subsystems and the work upwards

Top down testing:

Start with the high level , structure and interface and then work down

Testing with Simulator

Testing with a simulator allows testing a lot of features without risking the real system

- A simulator is only a model
- If code tested with a simulator needs a lot of rewriting to run on the real system some of the test advantages are lost
- A simulator allows repeating test under exactly the same conditions
- The simulator gives you ground truth
- It is possible to test emergency parts that you do not want to test in real life
- You can perform experiments faster allowing for more tests

Testing Systems Where Timing Matters

```
Interrupt:
{ tick++;
 if (tick==100){
   tick=0;sec++;
   if (sec==60){
     sec=0;min++;
     if (min==60){
       min=0;hour++;
       if (hour==24)
         hour=0;
     }
   }
 }
}
```

```
gettime(timetype *time){
  time->tick=tick;
  time->sec=sec;
  time->min=min:
  time->hour=hour;
 }
```

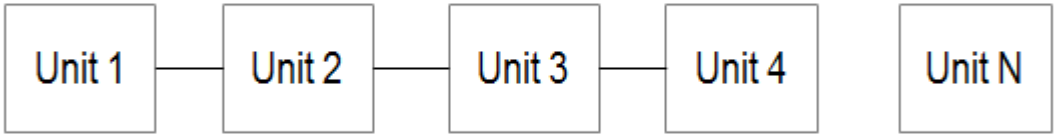
Debugging

Debugging is finding the root cause to an observed error and correct it.

General method:

1. Observe the error and collect all relevant data that can help
2. Try to find possible causes to the error
3. Make a test that can confirm the hypothesis or may narrow down the part of the system that can be responsable for the error

Debugging Series Structure

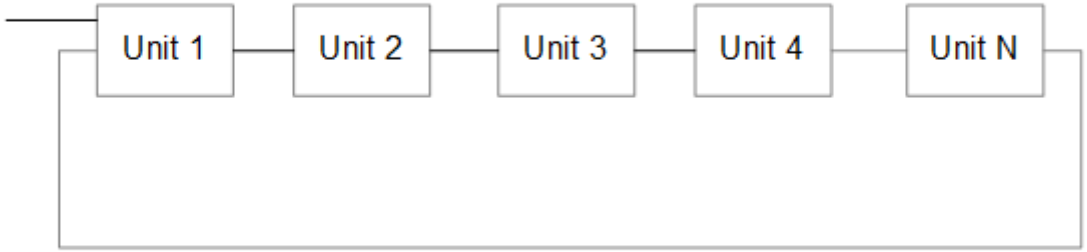


Error seen after unit N

Check after other modules e.g. using bisection

Continue until failing module found

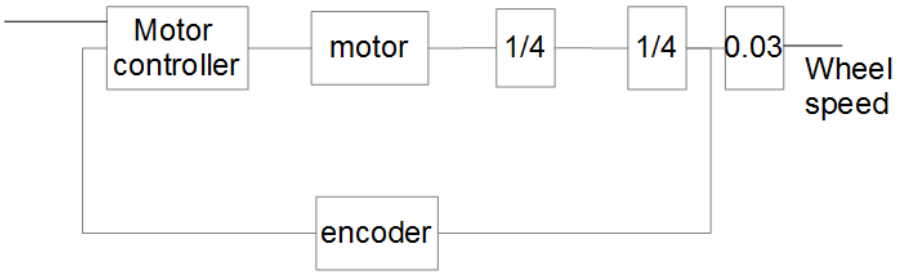
Debugging a Closeed Loop Structure

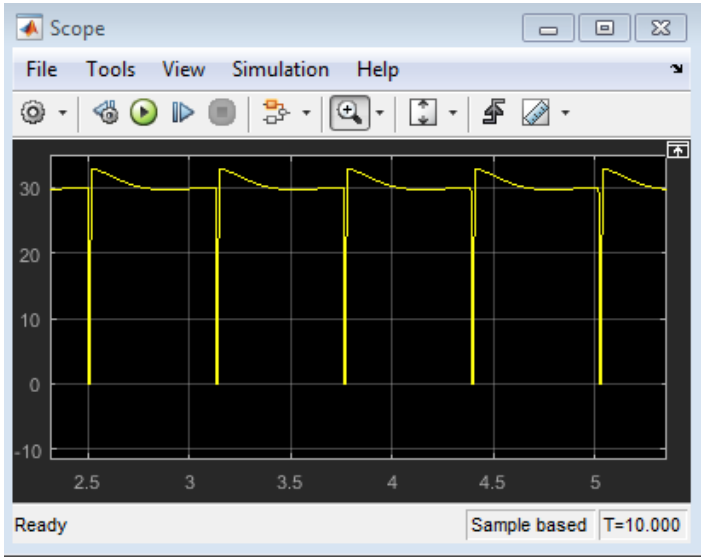


A signal will show up everywhere in the loop no matter which unit is the source

Probably necessary to open the loop to find error

Small Mobile Robot Speed Error





Minimizing the Number of Errors in the Code

- Make a clear specification of the problem and used algorithms
- Learn the language e.g. by studying programming examples from skilled programmers
- Use a good editor or IDE
- Turn on all warnings in the compiler
- Heed the warnings (no 'it's just a warning')
- Make comprehensive tests based on knowledge of the program structure
- Use reviews by other project members
- Explain the code to somebody else

Common Errors in C and C++ 1

1. The assignment operator ”=” confused with test operator ”==”

```
tal = 2
if (tal= 1) tal=-1;
printf(”tal = %d \n”, tal);
```

2. Arithmetic errors due to type mixing

```
double a;
unsigned int b,c;
a = 3/2;
printf(”a = %lf \n”, a);
b=10;
c=12;
a=b-c;
printf(”a = %lf \n”, a);
```

Common Errors in C and C++ 2

3. Pointer errors

```
void swap(int *a, int *b);

int *a , *b;
*a=1;
*b=2;
swap(a,b);
printf(”a = %d b=%d \n”, a,b);
```

4. Array error

```
int a[3];
```

a contains the elements a[0],a[1] a[2]
a[3] is a memory place outside the array!!!

Segmentation Error

Memory violation typically caused by array subscripting or pointer errors

Source file Segerr1.c

```
#include <stdio.h>

int main(int argc,char** argv){
int a,b,c,d,*e=0;
a=1;
b=2;
c=3;
d=4;
*e=5;
printf("a= %d b=%d c=%d d=%d *e=  %d\n",a,b,c,d,*e);
return 0;
}
```

Segmentation Error

Segmentation fault (core dumped)

ulimit –c unlimited	permit dumping of core
gdb segerr1 –c core	start debugger

```
Reading symbols from segerr1...done.
[New LWP 27158]
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/i386-linux-gnu/libthread_db.so.1".
Core was generated by `./segerr1'.
Program terminated with signal SIGSEGV, Segmentation fault.
#0  0x080484e2 in main (argc=1, argv=0xbf8ab824) at segerr1.c:9
9       *e=5;
(gdb)
```

Memmory Error

Source file array1.c

```
#include <stdio.h>
static int ar[4],j;
int main(int argc,char** argv){
int i,l;
if (argc < 2){
  printf("No argument \n");
  return(1);
}
sscanf(argv[1],"%d",&l);
j=100;
for (i=0;i<l;i++)
   ar[i]=i;
printf("j=  %d\n",j);
return 0;
}
```

Memmory Error

Source file scanferr.c

```
#include <stdio.h>

int main(int argc,char** argv){
int d=0;
printf("Enter integer\n");
if (scanf("%d",d)<1)
  printf("error\n");
printf("Integer is %d\n",d);
return 0;
}
```

Memmory Error

Source file scanferr.c

gdb output :

```
Reading symbols from scanferr...done.
[New LWP 5232]
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/i386-linux-gnu/libthread_db.so.1".
Core was generated by `./scanferr'.
Program terminated with signal SIGSEGV, Segmentation fault.
#0  0xb753b020 in _IO_vfscanf_internal (s=0xb7699c20 <_IO_2_1_stdin_>, format=format@er
1857    vfscanf.c: No such file or directory.
(gdb) bt
#0  0xb753b020 in _IO_vfscanf_internal (s=0xb7699c20 <_IO_2_1_stdin_>, format=format@er
#1  0xb7542156 in __isoc99_scanf (format=0x804862e "%d") at isoc99_scanf.c:37
#2  0x0804855e in main (argc=1, argv=0xbf891ad4) at scanferr.c:7
(gdb)
```

Exercise

Use putty to login to k385hn@tustin.iau.dtu.dk
 password smr4ever

Make a directory: mkdir mydir
Go to the new directory: cd mydir

Copy files:

```
cp /vhome/naa/test/*.c .
cp /vhome/naa/test/Makefile .
```

make array1 and run it 3 times with argument 3,4 and 5 explain results
make array2 and run it 3 times with argument 3,4 and 5 explain results
make and run segerr1 and scanferr and try debugger explain results